

CS 450: Assignment 05

Programming Assignments (95%)

- This assignment is in C++ and GLSL.
- **Setup:**
 - o Merge changes from the in-class repo into yours
 - I have included full **and updated** versions of the following files:
 - `src/include/student/VKImage.hpp`
 - `src/include/student/VKUniform.hpp`
 - `src/lib/student/VKImage.cpp`
 - `src/lib/student/VKUniform.cpp`
 - o Copy `src/app/Assign04.cpp` and name it `src/app/Assign05.cpp`
 - o Change the **application name** from "Assign04" to "Assign05"
 - o Change the **window title** to "Assign05: " + your SITNET ID
 - E.g., "Assign05: realemj"
 - o Copy folder `vulkanshaders/Assign04` as `vulkanshaders/Assign05`
 - o Modify `CMakeLists.txt` by adding the following line to the end of the file:
 - `CREATE_VULKAN_EXECUTABLE(Assign05)`
- **VKPipeline.hpp:**
 - o In BOTH `VulkanPipelineCreationInfo` AND `VulkanPipelineData`, add the following field:
 - `vector<vk::DescriptorSetLayout> allDescSetLayouts {};`
 - o In `createBasicVulkanPipeline()`, change this line to include the description layouts (also save the layouts for future cleanup):
 - `vk::PipelineLayoutCreateInfo pipelineLayoutInfo({},`
`creationInfo.allDescSetLayouts, creationInfo.pushConstantRanges);`
 - `data.allDescSetLayouts = creationInfo.allDescSetLayouts;`
 - o In `cleanupVulkanPipeline()`, add the following:
 - `for(int i = 0; i < pipelineData.allDescSetLayouts.size(); i++) {`
`vkInitData.device.destroyDescriptorSetLayout(`
`pipelineData.allDescSetLayouts.at(i));`
`}`
`pipelineData.allDescSetLayouts.clear();`

- **Assign05.cpp**
 - Add the following includes:
 - **#include "student/VKUniform.hpp"**
 - Add the following struct definitions:
 - **struct UBOVertex {**
 alignas(16) glm::mat4 viewMat;
 alignas(16) glm::mat4 projMat;
 };
 - **struct Camera {**
 glm::vec3 eye = glm::vec3(0.0f, 0.0f, 2.0f);
 glm::vec3 center = glm::vec3(0.0f, 0.0f, 1.0f);
 glm::vec3 up = glm::vec3(0.0f, 1.0f, 0.0f);
 };
 - **struct PointLight {**
 alignas(16) glm::vec4 pos = glm::vec4(1.5, 1.5, 1.5, 1.0);
 alignas(16) glm::vec4 vpos = glm::vec4(0,0,0,1.0);
 alignas(16) glm::vec4 color = glm::vec4(1.0, 1.0, 1.0, 1.0);
 };
 - **struct UBOFragment {**
 PointLight light;
 };
 - Modify your ForgeVertex to include a normal:
 - **struct ForgeVertex {**
 glm::vec3 pos;
 glm::vec4 color;
 glm::vec3 normal;
 };
 - Modify your UniformPush struct to include your normal transform:
 - **struct UniformPush {**
 alignas(16) glm::mat4 modelMat;
 alignas(16) glm::mat4 normMat;
 };
 - Add the following globals:
 - **UBOVertex uboVertHost {};**
 - **UBOFragment uboFragHost {};**
 - **Camera camera {};**
 - **glm::vec2 lastMousePos;**
 - Add a mouse motion callback:


```
static void mouse_position_callback(GLFWwindow* window,  
double xpos, double ypos)
```

- Get the current mouse position in pixels and get the DIFFERENCE in mouse movement:
 - `glm::vec2 mousePos = glm::vec2(xpos, ypos);`
 - `glm::vec2 diffPos = lastMousePos - mousePos;`
- Get the current framebuffer size and normalize the mouse motion difference:
 - `int width, height;`
 - `glfwGetFramebufferSize(window, &width, &height);`
 - `diffPos.x /= width;`
 - `diffPos.y /= height;`
- Multiply by speed:
 - `float speed = 50.0f;`
 - `diffPos *= speed;`
- Compute the current camera direction and local X axis:
 - `glm::vec3 direction = glm::normalize(camera.center - camera.eye);`
 - `glm::vec3 localX = glm::normalize(glm::cross(direction, camera.up));`
- Create the appropriate matrices:
 - `glm::mat4 negEye = glm::translate(-camera.eye);`
 - `glm::mat4 rotateY = glm::rotate(glm::radians(diffPos.x), camera.up);`
 - `glm::mat4 rotateX = glm::rotate(glm::radians(diffPos.y), localX);`
 - `glm::mat4 posEye = glm::translate(camera.eye);`
- Transform the camera "look at" point by 1) moving it by -eye, 2) rotating around the up axis, 3) rotating it around the local X axis, and 4) moving it back by +eye:
 - `camera.center = glm::vec3(posEye*rotateX*rotateY*negEye*glm::vec4(camera.center, 1.0));`
- Store the last known mouse position (IN PIXELS):
 - `lastMousePos = mousePos;`
- Modify key callback function:
 - At the beginning of the function, compute the camera direction, the local X direction, and the speed:
 - `glm::vec3 direction = glm::normalize(camera.center - camera.eye);`
 - `glm::vec3 localX = glm::normalize(glm::cross(direction, camera.up));`
 - `float speed = 0.1f;`
 - Add the following keys:
 - **GLFW_KEY_W:**
 - Add `direction*speed` to `camera.eye` AND `camera.center`
 - **GLFW_KEY_S:**
 - Subtract `direction*speed` from `camera.eye` AND `camera.center`
 - **GLFW_KEY_D:**
 - Add `localX*speed` to `camera.eye` AND `camera.center`

- **GLFW_KEY_A:**
 - Subtract localX*speed from camera.eye AND camera.center
- **GLFW_KEY_1:**
 - Set uboFragHost.light.color to white
- **GLFW_KEY_2:**
 - Set uboFragHost.light.color to red
- **GLFW_KEY_3:**
 - Set uboFragHost.light.color to green
- **GLFW_KEY_4:**
 - Set uboFragHost.light.color to blue
- Modify extractMeshData():
 - Change the vertex colors to yellow: **glm::vec3(1,1,0,1)**
 - Grab the normals from the Assimp mesh:
 - **auto nv = mesh->mNormals[i];**
 - **v.normal = glm::vec3(nv.x, nv.y, nv.z);**
- Change recordCommands() to be:


```
void recordCommands(VulkanInitData &vkInitData,
                    VulkanImage &depthImage,
                    uint32_t indexFIF,
                    uint32_t indexSwap,
                    vk::CommandBuffer &commandBuffer,
                    vk::QueryPool &queryPool,
                    VulkanPipelineData &pipelineData,
                    vector<VulkanMesh> &allMeshes,
                    UBOData &uboVertData,
                    UBOData &uboFragData,
                    vk::DescriptorSet &descriptorSet)
```

 - Add the depth attachment (calling a new function that creates the depth attachment for you):
 - **vk::RenderingInfoKHR ri{};**
 - **ri.setRenderArea(vk::Rect2D{ {0,0}, vkInitData.swapchain.extent })**
.setLayerCount(1)
.setColorAttachments(colorAtt)
.setPDepthAttachment(&createDepthAttachment(depthImage));
 - BEFORE rendering the meshes:
 - Compute the updated view and projection matrices based on the current camera information:
 - **uboVertHost.viewMat =**
glm::lookAt(camera.eye, camera.center, camera.up);
 - **float fov = glm::radians(90.0f);**

- **float aspectRatio =**
((float) vkInitData.swapchain.extent.width) / ((float)
vkInitData.swapchain.extent.height);
 - **float near = 0.01;**
 - **float far = 1000.0;**
 - **uboVertHost.projMat = glm::perspective(fov, aspectRatio,**
near, far);
 - **uboVertHost.projMat[1][1] *= -1;**
- Compute the normal transform (MAKE SURE TO DO THIS BEFORE THE CALL TO `commandBuffer.pushConstants()`):
 - **pv.normMat = glm::mat4(**
glm::transpose(
glm::inverse(
glm::mat3(uboVertHost.viewMat*modelMat))));
- Update the light's view position:
 - **uboFragHost.light.vpos =**
uboVertHost.viewMat*uboFragHost.light.pos;
- Copy over the UBO data and bind the descriptor set:
 - **copyToHostVisibleVulkanBuffer(**
vkInitData,
uboVertData.bufferData[indexFIF],
&uboVertHost);
 - **copyToHostVisibleVulkanBuffer(**
vkInitData,
uboFragData.bufferData[indexFIF],
&uboFragHost);
 - **commandBuffer.bindDescriptorSets(**
vk::PipelineBindPoint::eGraphics,
pipelineData.pipelineLayout,
0, descriptorSet, {});
- In your `main()` function:
 - Set the mouse motion callback:
 - **glfwSetCursorPosCallback(window, mouse_position_callback);**
 - Hide the cursor:
 - **glfwSetInputMode(window, GLFW_CURSOR,**
GLFW_CURSOR_DISABLED);
 - Initialize the last mouse position:
 - **double xpos, ypos;**
 - **glfwGetCursorPos(window, &xpos, &ypos);**
 - **lastMousePos = glm::vec2(xpos, ypos);**

- Add normal to attribute description list:
 - `pipelineCreateInfo.attribDesc.push_back(vk::VertexInputAttributeDescription(
 2, // location
 0, // binding
 vk::Format::eR32G32B32Sfloat, // format
 offsetof(ForgeVertex, normal) // offset
));`
- BEFORE we get to our pipeline creation, create UBOs with the appropriate number of frames-in-flight AND binding (function updated):
 - `UBOData uboVertData = createVulkanUniformBufferData(vkInitData, sizeof(UBOVertex), numberFramesInFlight, 0);`
 - `UBOData uboFragData = createVulkanUniformBufferData(vkInitData, sizeof(UBOFragment), numberFramesInFlight, 1);`
- Using the following new function, set the descriptor set layouts for the pipeline:
 - `pipelineCreateInfo.allDescSetLayouts = createDescriptorSetLayouts(vkInitData, {
 vk::DescriptorSetLayoutBinding(
 0, vk::DescriptorType::eUniformBuffer,
 1, vk::ShaderStageFlagBits::eVertex),
 vk::DescriptorSetLayoutBinding(
 1, vk::DescriptorType::eUniformBuffer,
 1, vk::ShaderStageFlagBits::eFragment)
 });`
- Using the following new function, create the descriptor pool:
 - `int uboCnt = 2;`
 - `vk::DescriptorPool descriptorPool = createDescriptorPool(vkInitData, uboCnt, numberFramesInFlight);`
- Using the following new function, create the descriptor sets:
 - `vector<vk::DescriptorSet> allDescriptorSets = createDescriptorSets(
 vkInitData,
 pipelineData.allDescSetLayouts,
 descriptorPool,
 numberFramesInFlight,
 {uboVertData, uboFragData}
);`
- Between the initial staging copy, create the depth images:
 - `vector<VulkanImage> allDepthImages {};`
 - `recreateAllVulkanDepthImages(vkInitData, stagingData.commandBuffer, allDepthImages);`

- Change the call to **recordCommands()**:
 - **recordCommands(vkInitData,**
 allDepthImages[indexFIF],
 indexFIF, indexSwap,
 commandData.perFIF[indexFIF].commandBuffer,
 queryPools.at(indexFIF),
 pipelineData,
 allMeshes,
 uboVertData,
 uboFragData,
 allDescriptorSets[indexFIF]);
 - Where the swapchain needs to be recreated, recreate the depth images:
 - **VulkanStagingData depthStageData =**
 beginStagingVulkanBufferCopies(vkInitData,
 commandData.commandPool);
 - **recreateAllVulkanDepthImages(vkInitData,**
 depthStageData.commandBuffer, allDepthImages);
 - **endStagingVulkanBufferCopies(vkInitData,**
 commandData.commandPool, depthStageData);
 - In the cleanup region:
 - **vkInitData.device.destroyDescriptorPool(descriptorPool);**
 - **cleanupVulkanUniformBufferData(vkInitData, uboFragData);**
 - **cleanupVulkanUniformBufferData(vkInitData, uboVertData);**
 - **cleanupAllVulkanDepthImages(vkInitData, allDepthImages);**
 - Create a UBOData instance for your vertex shader UBO:
- **Shader code:**
- **shader.vert:**
 - Add new input variable:
 - **layout(location = 2) in vec3 normal;**
 - Modify PushConstants to include the normal transform:
 - **layout(push_constant) uniform PushConstants {**
 mat4 modelMat;
 mat4 normMat;
 } **pc;**
 - Add the following structs:
 - **layout(set = 0, binding = 0) uniform UBOVertex {**
 mat4 viewMat;
 mat4 projMat;
 } **ubo;**

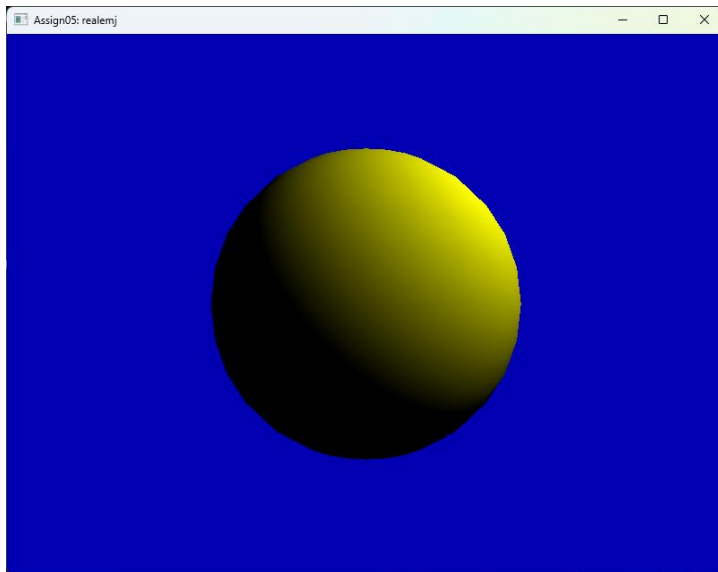
- Add new output variables:
 - **layout(location = 1) out vec3 interPos;**
 - **layout(location = 2) out vec3 interNormal;**
- In the main function:
 - Pass along the transformed normal to fragment shader:
 - **interNormal = mat3(pc.normMat)*normal;**
 - Compute the position after the model and view transform and ensure it is passed along to the fragment shader:
 - **vec4 pos = vec4(position, 1.0);**
 - **vec4 viewPos = ubo.viewMat*pc.modelMat*pos;**
 - **interPos = vec3(viewPos);**
 - Compute the final position after projection:
 - **gl_Position = ubo.projMat*viewPos;**
- **shader.frag:**
 - Add the following structs:
 - **struct PointLight {**
 vec4 pos;
 vec4 vpos;
 vec4 color;
 };
 - **layout(set = 0, binding = 1) uniform UBOFragment {**
 PointLight light;
 } ubo;
 - Add new input variables:
 - **layout(location = 1) in vec3 interPos;**
 - **layout(location = 2) in vec3 interNormal;**
 - In main function:
 - Renormalize normal:
 - **vec3 N = normalize(interNormal);**
 - Compute the light vector:
 - **vec3 L = normalize(vec3(ubo.light.vpos) - interPos);**
 - Calculate the diffuse lighting:
 - **vec3 finalColor =**
 vec3(ubo.light.color) * vec3(interColor) * max(0, dot(N, L));
 - Output the final color:
 - **out_color = vec4(finalColor, 1.0);**

Screenshot (5%)

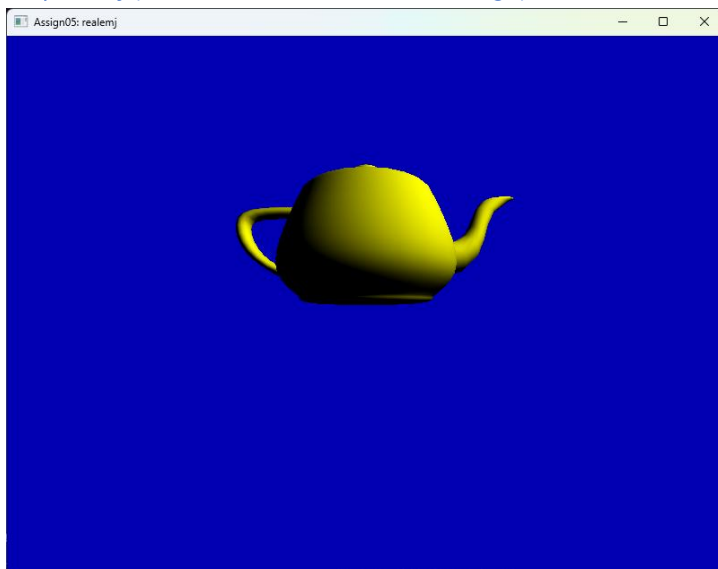
For this part of the assignment, **upload two screenshots** of the application window (note that window title should be "Assign05: yoursitnetid"):

- One displaying sphere.obj → Assign05_sphere.png
- One displaying teapot.obj → Assign05_teapot.png

sphere.obj (color need not match this image):



teapot.obj (color need not match this image):



Copy these images into the **screenshots** folder